scaling

Scaling-class testing (BOB-109 / §11.4.27)

Revision: 2 **Last modified:** 2026-08-21T21:18:59Z

Closes the `scaling` row that `docs/testing/test_type_matrix.md` recorded as **ABSENT** (“No scaling-tagged directory, test file, or HelixQA bank anywhere in the tree”), filed as **BOB-109**, the BOB-074 followup.

Why a second file next to `test_boba_scaling.py`

`tests/scaling/test_boba_scaling.py` was already tracked when this work started, landed by commit `12b439b`, whose own message records it as *“recover wave-6 subagent partial output before dispatch termination”*. Audited before writing anything new (§11.4.6 — establish the state, do not assume it):

Finding	Status
Fan-out axis assertion was a **tautology** — `other` is defined as the residual `N - accepted - rate_limited - timed_out`, then asserted to sum back to `N`	**FIXED** (this round)
SSE axis assertion reduced to `len(status_codes) == N`; caught a non-recording thread but **not a dead service**	**FIXED** (this round)
Docstring asserted the SSE route enforces `5/minute` “by contract” — never measured, and **false**	**CORRECTED** with the measurement
No p99 anywhere (§11.4.85 wants p50/p95/p99)	Addressed in the new file

Finding	Status
`scripts/run_all_challenges.sh:66` lists `scaling_horizontal_challenge.sh`, which **does not exist**	**OPEN** — reported, out of this scope

The tautology is not an inference. Pointing `MERGE_URL` at a closed port and invoking the test directly, it **passed against a dead service**:

```
[RED] pointing MERGE_URL at CLOSED port 39467 (service is DEAD)
[RED] RESULT: test PASSED against a DEAD service  <-- TAUTOLOGY CONFIRMED
```

Full transcript: `docs/qa/BOB-109/red_tautology_proof.txt`. That is §11.4.266 `green-but-broken` coverage-theater, so the file could not be cited as the coverage BOB-109 asks for. After the fix the same probe fails correctly (`merge service produced NO HTTP verdict at N=5`).

The scaling question actually asked

The merge service fans one query across three trackers in parallel and deduplicates the union. The dominant axis is **result-set size**, not user count — because the rate limiter binds long before the backend does (below). `Deduplicator.merge_results` pops a seed and rescans every remaining candidate, so all-distinct input — a broad query — is **quadratic by construction**.

Axis	What it measures	Limiter in the way?
A `TestDedupCostScaling`	dedup cost vs result-set size; complexity class, latency distribution, correctness at scale	No — offline, deterministic
B `TestRateLimitAdmissionEnvelope`	the admission ceiling itself, declared vs served	The limiter **is** the subject
C `TestLimiterFreeConcurrencyScaleOut`	concurrency scale-out where no	No — `:7189` has no limit headers

Axis	What it measures	Limiter in the way?
	limiter distorts it	

How the rate limiter was handled

Measured live 2026-08-21, not assumed:

- `:7187/` → `x-ratelimit-limit: 60` (dashboard class)
- `:7187/api/v1/theme/stream` → `120` (default class)
- `:7189/healthz` → **no limit headers at all**

Source of truth `api/rate_limit.py::DEFAULT_LIMITS` declares `search=10/minute`, `dashboard=60/minute`, `sse_stream=5/minute`, `default=120/minute`; `.env` sets no `RATE_LIMIT_*` override.

Two decisions follow, both deliberate:

1. **Concurrency scale-out is measured on `:7189`, not `:7187`.** Driving `/api/v1/search` past N=10 measures the limiter, not the fan-out. The limiter-free plane gives the service's own envelope.
2. **The limiter is never driven to exhaustion.** It is keyed per-IP and therefore **shared with every other agent on this host**; exhausting it would cross-contaminate their measurements (§11.4.119 single-resource-owner). The ceiling is read from the `x-ratelimit-*` response headers with **one request per class** — non-destructive, and still sufficient to detect a disabled or unwired limiter.

`x-ratelimit-limit` arrives as a **list** when several limits apply (observed `"60, 60"`); the binding ceiling is the minimum, so it is parsed as such (§11.4.201(9) field-identity — capacity list, not scalar).

Defect found by this axis — and a correction to how I first explained it

Two SSE routes, one rate-limit class (filed as **BOB-167**). `/api/v1/search/stream/{search_id}` carries `@_rl("sse_stream")` (`routes.py:801`) and serves `x-ratelimit-limit: 5`. Its sibling `/api/v1/theme/stream` (`routes.py:150`) carries **no limiter decorator at all** and falls through to the `default` class, serving **120**. Both are the same expensive shape: long-lived connections that pin a worker and a generator for their lifetime.

A CORRECTION WORTH KEEPING (§11.4.6). This doc, and the test, first explained the finding as *"`sse_stream` is declared and wired to nothing — `sse_limit_decorator` is applied nowhere"*. **That mechanism was wrong** and the coordinator rejected it. The wiring does not use a `*_limit_decorator` symbol at all; it uses `@_rl("sse_stream")`, and the class **is** applied — to `/search/stream`, verified here by direct measurement (5). The *measurement* that started this (120 on `/theme/stream`) was correct and is what makes the finding real; the *cause* was not. Searching for one symbol NAME and reading the zero hits as "wired nowhere" is the §11.4.201(7)(a) carrier/absence trap — the same class of error this suite exists to catch, committed by the suite's own author. Both halves are recorded so the next reader does not re-derive the wrong cause.

The test was rewritten to the **policy-neutral** invariant that survives the correction: two routes of the same SSE shape must serve the **same** class. Whether the right resolution is to classify `/theme/stream` or to widen `sse_stream` is an operator decision (BOB-167 acceptance (a)), and the test deliberately does not presume it — what it refuses to let pass silently is the divergence.

Per §11.4.238 this is also a **coverage escape**: it was found by this investigation, not by the automated regime — precisely the gap this file closes.

It is recorded as a `strict=True` xfail: a self-clearing record, not a suppression. It stays visible as `xfailed` in every report, and the moment both routes agree it flips to XPASS and **fails the run**, forcing the marker's removal. Evidence: `docs/qa/BOB-109/rate_limit_class_wiring.json` (`verdict: FAIL`, `tracked_as: BOB-167`).

Measured baselines

Every number below is quoted from `docs/qa/BOB-109/*.json` as committed, all written by a **single run** (`run_id 20260821T214914Z-pid1918585`). Each artifact carries its own `purpose`, `verdict` and `run_id` — see "Evidence contract".

These tables are **hand-transcribed** from those artifacts, and the transcription is not asserted — it is **checked**. See "Evidence contract" below for the mechanism and what defeated its first version.

An earlier revision of this doc quoted dedup p50 66.1/232.8/955.9/3308.1 with span 1.882, and healthz throughput "flat at ~885 req/s". Those numbers were real measurements from earlier runs in the authoring session, but the artifacts they came from had been overwritten by

later runs, so the doc cited figures its own evidence no longer contained. That is exactly the failure this section now prevents, and it is why the evidence contract below exists.

Dedup identity at scale — `dedup_identity_at_scale.json`

Ungated (runs on a busy host). N=400 distinct releases → 400 groups, **400 of 400 source releases preserved**, 0 missing, 0 invented, 0 cross-wired download binding(s). PASS.

Dedup collapse — `dedup_collapse_identical.json`

300 identical releases → **1 group aggregating all 300 sources**, 13.018 ms. PASS.

Dedup cost curve — NOT CURRENTLY CAPTURED

`dedup_cost_scaling.json` and `dedup_latency_distribution.json` are **absent by design**. Their tests are quiescence-gated (below) and this host has not been quiet enough to produce a valid measurement; the files that previously sat here were stale specimens from a contended run — one of them a span exponent of 2.1359, which **violates the file's own 2.10 gate**. Rather than commit a failing specimen as though it were a baseline, they are removed and the baseline is recorded as owed. See "Owed: one quiescent run".

Limiter-free scale-out — `healthz_scale_out_curve.json`

Captured at loadavg **22.97** (8 cpus), so the p99-ratio assertion was recorded but **not applied** (`p99_ratio_asserted: false`); the `ok == N` assertion applies always and passed at every rung.

N	workers	p50	p95	p99	throughput	ok
50	4	27.317 ms	63.734 ms	74.557 ms	107.7 req/s	50/50
100	8	35.735 ms	60.498 ms	89.923 ms	164.86 req/s	100/100
200	16	96.703 ms	242.854 ms	310.0 ms	111.83 req/s	200/200

Read this as a **contended-host** curve, not a clean baseline: every request still succeeded (350/350 across the ladder) and the p99 ratio across the ladder was 4.158x, inside the 25.0x ceiling — but the absolute throughput is depressed by co-tenant load and is **not** the service's ceiling. An earlier quiet-host run of the same ladder

recorded ~885 req/s with p50 3.65/6.61/11.99 ms; that artifact no longer exists, so it is reported here as context, explicitly **not** as evidence.

Admission envelope — `rate_limit_admission_envelope.json`

Binding limits read non-destructively, one request per class: `/` → **60**, `/api/v1/theme/stream` → **120**. PASS (limiter enforced, headers coherent).

SSE class divergence — `rate_limit_class_wiring.json`

`/api/v1/search/stream` → **5**, `/api/v1/theme/stream` → **120**, `divergent: true`, `verdict: FAIL`, `tracked_as: BOB-167`. This artifact is a **deliberately committed FAIL**: it is the evidence for the tracked defect, and it is labelled as such so no reader mistakes it for a passing baseline.

Evidence contract

Every artifact under `docs/qa/BOB-109/` self-describes:

- `purpose` — `baseline` or `mutation`
- `verdict` — `PASS` / `FAIL` (matrices roll up their per-N rows)
- `run_id` — identical across every file one process writes

Callers compute the pass condition **before** emitting, so a committed artifact is never silently a failing specimen.

These tables are **hand-transcribed** from those artifacts. Transcription drifts — it drifted twice while this doc was being written — so the agreement is not asserted, it is **checked**, by `tests/scaling/test_doc_evidence_agreement.py`: every decimal figure in this section must **equal a numeric value** in a committed artifact, the cited `run_id` must match the corpus, the corpus must be from one run, and each artifact must declare `purpose=baseline` with a PASS/FAIL verdict.

“Equal a value”, not “appear in the text”, is the load-bearing part. The first version of this checker compared figures as substrings of the raw JSON, and a reviewer defeated it by **rounding a table cell** — `26.131` retyped as `26.1` passed, because the rounded text is a prefix of the true value. Rounding while tidying a table is the likeliest transcription error there is, so the mechanism was blind to exactly the drift it exists to catch. Comparing against parsed numeric values closes it, along with a second channel where prose could “back” a figure (`11.4` extracted from a §-citation matching `§11.4.119` inside an artifact string).

It refuses to pass vacuously — an emptied section, a section thinned to a couple of live figures, or an empty corpus all fail rather than reporting agreement on nothing. The anti-vacuity floor counts only **live** (non-allowlisted) figures: it previously counted all figures against a floor of 12 while ten allowlisted ones sat in the section, an effective floor of two. Run it with the suite. It contains transcription drift; it does not prove the numbers correct or that the right figure was quoted for the right axis (§11.4.6).

Figures that legitimately appear here WITHOUT backing — the retracted ones below, the `2.10` source constant, and the quiet-host context figures — are enumerated with reasons in that file's `ALLOWED_UNBACKED`, and a further check fails if an exemption outlives the text it was granted for.

`_emit` refuses to write into `docs/qa/BOB-109/` when `BOBA_SCALING_MUTATION=1` is set without redirecting `EVIDENCE_DIR`. That guard exists because this directory has already carried experiment residue: a mutation harness once wrote a 20/40/60 ladder with a 443,190 ms p99 into `healthz_scale_out_curve.json`, and it was committed as evidence (via `7b45113`). §11.4.84's mutation-residue lesson, applied at the evidence layer.

Owed: one quiescent run

The two timing-derived tests have **never executed under their own precondition**. The ceiling is `loadavg/nproc <= 0.75` (6.0 on this 8-core host); every run recorded during this work sat at 9.2-40. The skip is loud and legitimate, but nothing currently ensures the gate ever runs — a §11.4.226 unexecuted-standing-guard.

Tracked as BOB-170: capture one quiescent GREEN run of `test_dedup_growth_does_not_regress_past_quadratic` and `test_dedup_latency_distribution_at_operating_size`, which would also regenerate the two absent artifacts above and close the honest gap recorded next to `MAX_GROWTH_EXPONENT` in the source. When that run lands, these tables need re-transcribing — and the checker above is what will catch it if they are not.

Why the gate reads a span exponent, not per-step

Per-step exponents are noise-dominated on a host shared with other agents. Measured during this work: a co-tenant load spike produced a `0.981` step, and extending the ladder to N=1600

pushed the **baseline** to `2.328` — a longer lever arm made the estimator *noisier*, not sharper, so the 1600 rung was dropped (§11.4.201(6)).

The span exponent is ratio-based and therefore far more stable than per-step: three baseline runs gave 1.832 / 1.848 / 1.759, and a run under load average 11.63 still read 1.882. It is **more robust, not load-invariant** — see the validity section below, which is why the timing tests are quiescence-gated.

Threshold chosen from measured mutant separation, injecting an $O(n^3)$ comparator:

Injected cubic term	Span exponent	Caught at 2.10?
none (baseline)	1.76 – 1.88	passes, as it must
`n/4` ops per compare	1.986	**no**
`n/2` ops per compare	2.145	yes
`n` ops per compare	2.450	yes

Honest sensitivity floor (§11.4.6): this gate catches a cubic term whose per-comparison cost is ≥ ~`n/2` elementary ops. A cubic term 4× smaller is indistinguishable from quadratic within N ≤ 800 on this host — its crossover lies beyond the tested range. Stated blind spot, not a claim of completeness.

The metric is only valid on a quiet host (§11.4.201(8))

The 2.10 threshold was validated, then **invalidated by measurement**, then re-scoped. Shipping it unconditionally would have been a false-positive refusal generator. Measured on this shared workstation:

loadavg_1m (8 cpus)	baseline span exponent	verdict at 2.10
~2.8 – 9	1.759 / 1.832 / 1.848	passes correctly
~9.4 – 11.6	1.882	passes correctly
~11.1	**2.136**	**FALSE FAIL on correct code**
~15 – 23	up to **2.358**	false fail

And at load 15–23 the injected-cubic mutants read **2.278 – 2.331** — *below* the loaded baseline. Under contention **the signal is smaller than the noise and the two are not separable in either direction**. Best-of-N instead of median does not rescue it

(baseline 1.857 / 1.904 / 2.309 vs cubic 2.278 / 2.294): sustained contention degrades the largest rung disproportionately, which is exactly what an exponent measures.

Per §11.4.201(8) a metric whose *correct* end-state can cross its own threshold is invalid and must not gate work. So the two timing-derived tests now require `loadavg_1m / nproc <= 0.75` and otherwise **SKIP-with-reason**, printing the resolved numbers (§11.4.201(5)):

```
SKIP-OK BOB-109 §11.4.201(8): host not quiescent — loadavg_1m
18.09
over 8 cpus = 2.26 per cpu > 0.75. A timing-derived complexity
exponent
is not separable from contention noise at this load; emitting a
verdict
here would be a coin flip in both directions.
```

What actually survives a skip — stated precisely, because an earlier revision of this paragraph overclaimed. The ladder-rung identity checks live *inside* the two quiescence-gated tests, so on a busy host they do **not** run. What runs regardless of load is:

- `test_dedup_identity_preserved_at_scale` — **ungated**, N=400 distinct releases, asserts the merged view still contains every input release. This test exists precisely so the at-scale distinct-input jurisdiction is not lost when the timing tests skip.
- `test_dedup_collapses_identical_union_to_one_group` — ungated, and now also asserts the survivor aggregated all 300 sources.

Both assert identity, never elapsed time, so contention cannot make them lie in either direction.

Boundary stated honestly (§11.4.6): the growth gate's GREEN polarity was observed at loadavg 9.4–11.6 (span 1.882) and in three stability runs (1.759 / 1.832 / 1.848), all well under 2.10. It was **not** observed under the new `<= 0.75/cpu` precondition, because the host did not become that quiet while this work ran and other agents' load is not mine to clear. Quiescence is strictly more favourable than the conditions those passes were measured under, so the gate passing there follows from the data — but it is an inference from measurement, not a run I can paste.

Anti-bluff properties

- **Every shipped assertion is observed to fail under a mutation** (§11.4.115). The earlier "8/8" in this section was stale — the assertion set grew, and two of the new ones (`total == n`, `aggregated == n`) had no recorded RED until a reviewer supplied it. Full transcript: `docs/qa/BOB-109/red_tautology_proof.txt`.

Assertion	Mutation that makes it fail
growth `span_exponent <= 2.10`	injected O(n³) comparator (2.2213 / 2.3851)
identity `len(groups) == n`	no-op dedup returning `[]`
identity `links == expected`	tail-truncation (last 10% → dup of group 0)
identity `total == n`	sources duplicated (799 rows for 400 inputs)
identity `not mismatched`	`download_urls` cross-wired (399 of 400 groups)
identity `url_union == expected`	one group's binding cleared (1 release unclickable)
distribution `p99 < budget`	inflated p99
collapse `len(groups) == 1`	dedup that never matches
collapse `aggregated == n`	survivor drops sources (kept 150 of 300)
collapse `not mismatched`	survivor offers a foreign torrent
collapse `url_union == expected`	survivor offers nothing
limiter enforced + coherent	probe a service with no limit headers
SSE class consistency	fails today — it IS the BOB-167 defect
healthz `ok == N`	dead service
healthz p99 ratio	inflated top-rung p99
SSE / fan-out `accepted + 429 >= 1`	dead service; and wedged service (fails at the timeout cap)
doc-evidence checker (5 tests)	8 mutations incl. both vacuity cases

The growth gate is additionally observed to **pass** on correct code, so it is not a §11.4.201(1) false-positive refusal.

Honest exception: `len(results) == n_parallel` in the fan-out axis has no independent mutant. It asserts the thread pool returned one result per submission; breaking it would mean breaking `ThreadPoolExecutor` itself. It is listed here rather than counted as proven.

- **Timing is tied to real work, and identity to the user.** Each timed dedup asserts group count, the SET of preserved releases, no duplicated sources, and that every group's `download_urls` come from its OWN sources. Count alone was not

enough: a mutation preserving the count while cross-wiring `download_urls` passed every earlier guard while 399 of 400 rows would have handed the user the wrong torrent. `MergedResult.to_dict()` serves `download_urls` — it is the product's download binding, so identity of the inputs alone left the user-visible half unguarded.

- **Correctness at scale** is asserted separately and **ungated** (300 identical releases collapse to 1 group carrying all 300 sources, and 400 distinct releases keep their identity), so a dedup that stopped matching cannot pass by getting quicker — and a busy host still gets real at-scale coverage when the timing tests skip.
- **Host safety is a precondition, not an afterthought** (§12.6/§12.12). The suite reads memory %, thread headroom, CPU count and load average, refuses to scale outside them with the resolved numbers printed (§11.4.201(5)), caps workers at 16 on this 8-core host, and records the reading inside every evidence artifact.
- **Cleanup on every exit path** (§11.4.14). Every executor is a context manager (joins on all paths) and every response closes in a `finally`; a session-scoped finaliser fails the run if a pool leaks.

Running it

```
python3 -m pytest tests/scaling/ -v --import-mode=importlib -p
        no:schemathesis
```

`-p no:schemathesis` is required on this host for an unrelated pre-existing reason: the plugin fails to import (`No module named 'rpds.rpds'`) and takes the whole pytest session down with it.

If you instead disable plugin autoloading wholesale, **`-p timeout` is mandatory** — otherwise `pytest.mark.timeout` is silently inert and the per-test timeouts this suite relies on do not apply (a `--strict-config` run also needs the addopts' `--timeout` to have a provider):

```
PYTEST_DISABLE_PLUGIN_AUTOLOAD=1 python3 -m pytest tests/scaling/
        -v \
  --import-mode=importlib -p timeout -p randomly
```

Under `nice`/`ionice` on a shared host:

```
nice -n 19 ionice -c 3 python3 -m pytest tests/scaling/ -q \
  --import-mode=importlib -p no:schemathesis
```

Mutation/experiment runs MUST set `BOBA_SCALING_MUTATION=1` **and** redirect `EVIDENCE_DIR` to a scratch path; with the flag set and no redirect, `_emit` refuses to write rather than overwrite the

committed corpus. Both sides of that comparison are `.resolve()`d — `Path.__eq__` is lexical, so a relative-join or symlinked path compared unequal and a reviewer used exactly that to slip a mutation artifact into the committed corpus before the guard was hardened.

After any full-suite run the artifacts are regenerated with a new `run_id`, so `test_doc_evidence_agreement.py` will FAIL until these tables are re-transcribed. That is the mechanism working, not a broken suite — it caught its first real drift on its own first full run. Re-transcribe the measured-baselines section from `docs/qa/BOB-109/*.json` and re-run. Running the checker alone never regenerates anything.

Axis A runs anywhere. Axes B and C SKIP-with-reason when `:7187` / `:7189` are down — they never start the stack.